Sized simultaneous cut elimination for non-wellfounded intersection subtyping with fixpoints

Olivier Laurent, Jui-Hsuan Wu

github:olaure01/islogic:FSCD2026; GNU GPL version3 (native author proof source; no relicensing as article CC BY)

Editorial note. Official paper: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2026.22. Selected native proof: https://github.com/olaure01/islogic/tree/FSCD2026. Copyright Olivier Laurent and Jui-Hsuan Wu, GPLv3 as supplied in the repository. This is an unexecuted native author proof package, not a proof-assistant validation report. Full source retains later unrelated Admitted bridges, which are not the selected core cut theorem and are not additional counted results.

Allowed standard prerequisites (editorial).

- Rocq9.1 standard-library List/Wf_nat/Lia/CMorphisms; dependent pairs, structural/well-founded induction and coinductive derivations. Exact native imports and repo README/_RocqProject/configure are retained. There are no external nonstandard imported source files in this tagged repository.

```
(** Cut elimination property for an entailment relation coming     *)
(** from an extension of BCD intersection subtyping with equations *)
(** using non-wellfounded derivations                              *)

(** Thanks to Damien Pous for his support on [CoInductive] and [CoFixpoint] *)

From Stdlib Require Import List Wf_nat Lia CMorphisms.
Import ListNotations.

Set Default Goal Selector "!".
Set Default Proof Using "Type".
Set Implicit Arguments.
Unset Printing Use Implicit Types.
Generalizable All Variables.


(** * Preliminaries *)

Notation "l · a" := (l ++ a :: nil) (at level 55).

Theorem rev_rect A (P : list A -> Type) : P nil -> (forall x l, P l -> P (l · x)) -> forall l, P l.
Proof.
intros Hnil Hcons l. rewrite <- (rev_involutive l).
exact (list_rect (fun z => P (rev z)) Hnil (fun _ _ => Hcons _ _) _).
Qed.

Infix "<=>" := iffT (at level 95, no associativity).

Inductive cclos_refl_trans A (R : crelation A) : crelation A :=
| crt_step [x y] : R x y -> cclos_refl_trans _ x y
| crt_refl {x} : cclos_refl_trans _ x x
| crt_trans [x y z] : cclos_refl_trans _ x y -> cclos_refl_trans _ y z -> cclos_refl_trans _ x z.

Lemma PreOrder_cclos `{HT : PreOrder A R} a b : cclos_refl_trans R a b -> R a b.
Proof. intro Hc. induction Hc; [ | reflexivity | etransitivity ]; eassumption. Qed.

Instance cclos_PreOrder A (R : crelation A) : PreOrder (cclos_refl_trans R).
Proof. split; [ exact (@crt_refl _ _) | exact (@crt_trans _ _) ]. Qed.


(** * Formulas *)

Section Formulas.

(** Atoms *)
Variable A : Type.

(** Formulas over the set of atoms [A] *)
(**   with one constant Ω = [ovar None] *)
Inductive form := ovar (_ : option A) | inter (_ _ : form) | arrow (_ _: form).
Notation Ω := (ovar None).
```

```
Infix ∩"" := inter (at level 35).
Infix →"" := arrow (at level 41, right associativity).

Fixpoint fsize a := S
match a with
| ovar _ => 0
| b ∩ c | b → c => fsize b + fsize c
end.

(** Intersections of arrows *)
Reserved Notation ⋂→"( a )" (at level 0, format ⋂→"( a )").
Inductive ia_formula : form -> Type :=
| ia_omega : ⋂→Ω()
| ia_arrow a b : ⋂→(a → b)
| ia_inter a b : ⋂→(a) -> ⋂→(b) -> ⋂→(a ∩ b)
where ⋂→"( a )" := (ia_formula a).

(** Arrow components of intersections (of arrows) *)
Reserved Notation "a → b ∈ c" (at level 50, b at next level, c at next level).
Inductive left_right_ia : form -> form -> form -> Type :=
| lria_arrow a b : a → b ∈ a → b
| lria_inter1 a b c d : a → b ∈ c -> ⋂→(d) -> a → b ∈ c ∩ d
| lria_inter2 a b c d : a → b ∈ c -> ⋂→(d) -> a → b ∈ d ∩ c
where "a → b ∈ c" := (left_right_ia a b c).

(** Intersection components of intersections of arrows *)
(** as used in conclusion of beta-condition *)
Reserved Notation "a → b ∈⋂ c" (at level 50, b at next level, c at next level).
Inductive left_right_iia : form -> form -> form -> Type :=
| lriia_lria a b c : a → b ∈ c -> a → b ∈⋂ c
| lriia_omega c : ⋂→(c) -> Ω → Ω ∈⋂ c
| lriia_inter a b c d e : a → c ∈⋂ e -> b → d ∈⋂ e -> (a ∩ b) → (c ∩ d) ∈⋂ e
where "a → b ∈⋂ c" := (left_right_iia a b c).

Lemma rmon_lriia d e a c : d → e ∈⋂ a -> ⋂→(c) -> (d → e ∈⋂ (a ∩ c)) * (d → e ∈⋂ (c ∩ a)).
Proof.
intro H. induction H as [ | | ? ? ? ? ? ? IH1 ? IH2] in c |- *; intros.
- split; apply lriia_lria; [ apply lria_inter1 | apply lria_inter2 ]; assumption.
- split; apply lriia_omega; constructor; assumption.
- split; (apply lriia_inter; [ apply IH1 | apply IH2 ]); assumption.
Qed.

End Formulas.

Notation Ω := (ovar None).
Infix ∩"" := inter (at level 35).
Infix →"" := arrow (at level 41, right associativity).
Notation ⋂→"( a )" := (ia_formula a) (at level 0, format ⋂→"( a )").
Notation "a → b ∈⋂ c" := (left_right_iia a b c) (at level 50, b at next level, c at next level).


(** * Subtyping *)

(** ** Subtyping parameters: primitive order ≺ on atoms and function δ for fixpoint equations *)

Record subtyping_data := {
  At :> Type;
  subat : crelation At;
  valueat : At -> @form At }.
Notation δ x := (valueat _ x).
Infix ≺"" := (subat _) (at level 50).
Infix ≺"*" := (cclos_refl_trans (subat _)) (at level 50).

Coercion var [ST] (u : ST.(At)) := @ovar ST (Some u).

(** ** Subtyping systems *)

(** Finite partial derivations with leaves [var_rel] given by [LF] *)
(**   [nat] parameters control the number of unfolding rules [var_right] and [var_left] *)
(** [fsub (cclos_refl_trans (subat _))] is <IS *)
Inductive fsub {ST} {LF : crelation ST.(At)} :
  nat -> nat -> nat -> @form ST -> list (@form ST) -> @form ST -> Type :=
| omega_right a l : fsub 0 0 0 a l Ω
| inter_left1 a b c l p q r : fsub p q r a l c -> fsub p q r (a ∩ b) l c
| inter_left2 a b c l p q r : fsub p q r a l c -> fsub p q r (b ∩ a) l c
| inter_right a b c l p1 q1 r1 p2 q2 r2 : fsub p1 q1 r1 c l a -> fsub p2 q2 r2 c l b ->
    fsub (min p1 p2) (max q1 q2) (max r1 r2) c l (a ∩ b)
| arrow_left a b c d l p1 q1 r1 p2 q2 r2 : fsub p1 q1 r1 c nil a -> fsub p2 q2 r2 b l d ->
    fsub (p1 + p2) (q1 + q2) (r1 + r2) (a → b) (c :: l) d
| arrow_right a b c l p q r : fsub p q r c (l · a) b -> fsub p q r c l (a → b)
| var_rel x y : LF x y -> fsub 0 0 0 x nil y (* leaf *)
| var_right x c l p q r : fsub p q r c l δ( x) -> fsub p q (S r) c l x
| var_left x c l p q r : fsub p q r δ( x) l c -> fsub (S p) (S q) r x l c.
Arguments omega_right {_ _ _ _}, [_ _] _ _.
Arguments inter_left1 [_ _ _ _ _ _ _ _ _] _,  [_ _ _ _] _ [_ _ _ _] _.
Arguments inter_left2 [_ _ _ _ _ _ _ _ _] _,  [_ _ _ _] _ [_ _ _ _] _.
Arguments arrow_right [_ _ _ _ _ _ _ _ _] _, [_ _ _ _ _] _ [_ _ _] _.
Arguments var_rel [_ _ _ _].
Arguments var_right [_ _ _ _ _ _ _ _] _, [_ _] _ [_ _ _ _ _] _.
Arguments var_left [_ _ _ _ _ _ _ _] _, [_ _] _ [_ _ _ _ _] _.
Arguments fsub [_] _.
```

```
Notation ISat := (fsub (cclos_refl_trans (subat _))).

Definition fsub_funct ST (LF1 LF2 : crelation ST.(At)) (H12 : forall u v, LF1 u v -> LF2 u v) a l b p q r :
  fsub LF1 p q r a l b -> fsub LF2 p q r a l b.
Proof.
revert a l b p q r. fix IH 7. intros a l b p q r pi.
destruct pi; try (constructor; apply IH; assumption).
constructor. apply H12. assumption.
Defined.


(** Finite partial derivations with no unfolding: on formulas *)
Definition fsub_0 {ST} : crelation ST.(At) -> crelation (@form ST) := fun LF a b => fsub LF 0 0 0 a nil b.
(** Finite partial derivations with one [var_right] and one [var_left] on each branch: on atoms *)
Definition fsub_1 {ST} : crelation ST.(At) -> crelation ST.(At) := fun LF x y => fsub LF 1 1 1 x nil y.

(** Non-wellfounded valid derivations of conclusions of checkpoints (to be used as premises of [var_rel]) *)
(** checkpoints contain a transitive closure (a.k.a multi-cut) *)
CoInductive isub {ST} (x y : ST.(At)) : Type :=
| check_point : x <* y -> cclos_refl_trans (fsub_1 isub) x y -> isub x y.

(** Non-wellfounded valid derivations: system <δIS *)
Definition ISeq {ST} := @fsub ST isub.

Instance isub_PreOrder {ST} : PreOrder (@isub ST).
Proof.
split.
- intro x. constructor; reflexivity.
- intros x y z Hi1 Hi2. destruct Hi1, Hi2. constructor; econstructor; eassumption.
Qed.

(** ** Axiom expansion *)

Instance fsub0_Reflexive {ST} `{HR : Reflexive _ R} : Reflexive (@fsub_0 ST R).
Proof.
unfold Reflexive. fix IH 1. intro a.
destruct a as [ [ u | ] | a1 a2 | a1 a2 ].
- apply var_rel. reflexivity.
- apply omega_right.
- apply (@inter_right _ _ _ _ _ _ 0 0 0 0 0 0); [ apply inter_left1, IH | apply inter_left2, IH ].
- apply arrow_right, (@arrow_left _ _ _ _ _ _ _ 0 0 0 0 0 0); apply IH.
Defined.


Section ProofTheory.

Context {ST : subtyping_data}.
Notation formula := (@form ST).

Implicit Type x y : ST.
Implicit Type a b c : formula.

(** (Finite) Prefix of a derivation *)
(**    with leaves in <* *)
(**    ie as derivations in <IS *)
Fixpoint prefix a l b p q r (pi : ISeq p q r a l b) : ISat p q r a l b :=
match pi with
| omega_right => omega_right
| inter_left1 pi1 => inter_left1 (prefix pi1)
| inter_left2 pi1 => inter_left2 (prefix pi1)
| inter_right pi1 pi2 => inter_right (prefix pi1) (prefix pi2)
| arrow_left pi1 pi2 => arrow_left (prefix pi1) (prefix pi2)
| arrow_right pi1 => arrow_right (prefix pi1)
| var_rel ipi => match ipi with check_point s _ => var_rel s end
| var_right pi => var_right (prefix pi)
| var_left pi => var_left (prefix pi)
end.


(** ** Sizes *)

(** number of rules ([max] on slices) *)
Fixpoint psize R a l b p q r (pi : fsub R p q r a l b) := S
match pi with
| omega_right | var_rel _ => 0
| inter_left1 pi1 | inter_left2 pi1 | arrow_right pi1 | var_right pi1 | var_left pi1 => psize pi1
| arrow_left pi1 pi2 => psize pi1 + psize pi2
| inter_right pi1 pi2 => max (psize pi1) (psize pi2)
end.

Lemma psize_prefix a l b p q r (pi : ISeq p q r a l b) : psize (prefix pi) = psize pi.
Proof. induction pi; cbn; auto. destruct l. reflexivity. Qed.

(** number of [var_left] and [var_right] rules ([max] on slices) *)
Fixpoint vsize R a l b p q r (pi : fsub R p q r a l b) :=
match pi with
| var_right pi1 | var_left pi1 => S (vsize pi1)
| omega_right | var_rel _ => 0
| inter_left1 pi1 | inter_left2 pi1 | arrow_right pi1 => vsize pi1
| arrow_left pi1 pi2 => vsize pi1 + vsize pi2
```

```
| inter_right pi1 pi2 => max (vsize pi1) (vsize pi2)
end.

Lemma vsize_min_max R a l b p q r (pi : fsub R p q r a l b) : max q (p + r) <= vsize pi <= q + r.
Proof. induction pi; simpl vsize; lia. Qed.

Lemma vsize_prefix a l b p q r (pi : ISeq p q r a l b) : vsize (prefix pi) = vsize pi.
Proof. induction pi; cbn; auto. destruct l. reflexivity. Qed.


(** ** Invertibility *)

Lemma inv_inter_right R c l a b p q r (pi : fsub R p q r c l (a ∩ b)) :
  {'(p', q', r') & { pi' : fsub R p' q' r' c l a & psize pi' < psize pi & vsize pi' <= vsize pi }}
* {'(p', q', r') & { pi' : fsub R p' q' r' c l b & psize pi' < psize pi & vsize pi' <= vsize pi }}.
Proof.
remember (a ∩ b) as d eqn:Hd. induction pi in a, b, Hd |- *; destr_eq Hd; subst.
- destruct (IHpi _ _ eq_refl) as [[[[pl ql] rl] [pil' Hpl Hvl]] [[[pr qr] rr] [pir' Hpr Hvr]]]. split.
  + exists (pl, ql, rl), (inter_left1 pil'); cbn; lia.
  + exists (pr, qr, rr), (inter_left1 pir'); cbn; lia.
- destruct (IHpi _ _ eq_refl) as [[[[pl ql] rl] [pil' Hpl Hvl]] [[[pr qr] rr] [pir' Hpr Hvr]]]. split.
  + exists (pl, ql, rl), (inter_left2 pil'); cbn; lia.
  + exists (pr, qr, rr), (inter_left2 pir'); cbn; lia.
- split.
  + exists (p1, q1, r1), pi1; cbn; lia.
  + exists (p2, q2, r2), pi2; cbn; lia.
- destruct (IHpi2 _ _ eq_refl) as [[[[pl ql] rl] [pil' Hpl Hvl]] [[[pr qr] rr] [pir' Hpr Hvr]]]. split.
  + exists (p1 + pl, q1 + ql, r1 + rl), (arrow_left pi1 pil'); cbn; lia.
  + exists (p1 + pr, q1 + qr, r1 + rr), (arrow_left pi1 pir'); cbn; lia.
- destruct (IHpi _ _ eq_refl) as [[[[pl ql] rl] [pil' Hpl Hvl]] [[[pr qr] rr] [pir' Hpr Hvr]]]. split.
  + exists (S pl, S ql, rl), (var_left pil'); cbn; lia.
  + exists (S pr, S qr, rr), (var_left pir'); cbn; lia.
Qed.

Lemma inv_arrow_right R c l a b p q r (pi : fsub R p q r c l (a → b)) :
  {'(p', q', r') & { pi' : fsub R p' q' r' c (l · a) b & psize pi' < psize pi & vsize pi' <= vsize pi }}.
Proof.
remember (a → b) as d eqn:Hd. induction pi in a, b, Hd |- *; destr_eq Hd; subst.
- destruct (IHpi _ _ eq_refl) as [[[p' q'] r'] [pi' Hp Hv]].
  exists (p', q', r'), (inter_left1 pi'); cbn; lia.
- destruct (IHpi _ _ eq_refl) as [[[p' q'] r'] [pi' Hp Hv]].
  exists (p', q', r'), (inter_left2 pi'); cbn; lia.
- destruct (IHpi2 _ _ eq_refl) as [[[p' q'] r'] [pi' Hp Hv]].
  exists (p1 + p', q1 + q', r1 + r'). exists (arrow_left pi1 pi'); cbn; lia.
- exists (p, q, r), pi; cbn; lia.
- destruct (IHpi _ _ eq_refl) as [[[p' q'] r'] [pi' Hp Hv]].
  exists (S p', S q', r'), (var_left pi'); cbn; lia.
Qed.

Lemma inv_var_right R c y p q : fsub R p q 1 c nil y -> fsub R p q 0 c nil δ( y).
Proof.
intro pi.
remember (var y) as d eqn:Hd. remember nil as l eqn: Hl. remember 1 as r eqn:Hr.
induction pi in y, Hd, Hl, Hr |- *; destr_eq Hd; destr_eq Hl; destr_eq Hr; subst.
- apply inter_left1, IHpi; reflexivity.
- apply inter_left2, IHpi; reflexivity.
- assumption.
- apply var_left, IHpi; reflexivity.
Qed.

Lemma inv_var_left R x l d r : fsub R 1 1 r x l d -> fsub R 0 0 r δ( x) l d.
Proof.
intro pi.
remember (var x) as c eqn:Hc. remember 1 as p eqn:Hp. remember 1 as q eqn:Hq.
rewrite Hp in pi at 2. rewrite Hq in Hp.
induction pi in x, Hc, Hp, Hq |- *; destr_eq Hc; destr_eq Hp; destr_eq Hq; subst.
- change 0 with (min 0 0) at 1. change 0 with (max 0 0) at 3. apply inter_right.
  + assert (Hs1 := vsize_min_max pi1). apply (IHpi1 _ eq_refl); lia.
  + assert (Hs2 := vsize_min_max pi2). apply (IHpi2 _ eq_refl); lia.
- apply arrow_right, IHpi; reflexivity.
- apply var_right, IHpi; reflexivity.
- assumption.
Qed.

Lemma var_var_unfold R (x y : ST) : fsub_0 R δ( x) δ( y) <=> fsub_1 R x y.
Proof.
split; intro pi.
- apply var_right, var_left, pi.
- apply inv_var_right, inv_var_left, pi.
Qed.


(** * Transitivity elimination *)




Theorem sub_trans_size [qr b s] `{HRt : PreOrder _ R} : (0 < qr -> R = @isub ST) ->
  (forall a c d l1 l2 p1 p2 q1 q2 r1 r2
     (pi1 : fsub R p1 q1 r1 a nil b) (pi2 : fsub R p2 q2 r2 c (l1 ++ b :: l2) d),
```

```
     vsize pi1 + vsize pi2 = qr -> psize pi1 + psize pi2 = s ->
     {'(p, q, r) & { pi : fsub R p q r c (l1 ++ a :: l2) d & vsize pi <= qr }})
* (forall a c l1 l2 p1 p2 q1 q2 r1 r2 (pi1 : fsub R p1 q1 r1 a l1 b) (pi2 : fsub R p2 q2 r2 b l2 c),
     vsize pi1 + vsize pi2 = qr -> psize pi1 + psize pi2 = s ->
     {'(p, q, r) & { pi : fsub R p q r a (l1 ++ l2) c & vsize pi <= qr }}).




Proof.
induction qr as [qr IHqr] using (well_founded_induction_type lt_wf) in b, s |- *. intro HR.
remember (fsize b) as sb eqn:Hsb.
induction sb as [sb IHb] using (well_founded_induction_type lt_wf) in b, s, Hsb |- *.
induction s as [s IHs] using (well_founded_induction_type lt_wf) in b, Hsb |- *. subst sb.

assert (forall m, m < qr -> 0 < m -> R = @isub ST) as HR' by (intros; apply HR; lia).

assert (forall a b' c d l1 l2 p1 p2 q1 q2 r1 r2
              (pi1 : fsub R p1 q1 r1 a nil b') (pi2 : fsub R p2 q2 r2 c (l1 ++ b' :: l2) d),
        ((vsize pi1 + vsize pi2 < qr)%type
       + (vsize pi1 + vsize pi2 <= qr /\ fsize b' < fsize b)
       + (vsize pi1 + vsize pi2 <= qr /\ fsize b' <= fsize b /\ psize pi1 + psize pi2 < s)) ->
       {'(p, q, r) & { pi : fsub R p q r c (l1 ++ a :: l2) d & vsize pi <= vsize pi1 + vsize pi2 }})
  as IH1.
{ intros a b' c d l1 l2 p1 p2 q1 q2 r1 r2 pi1 pi2 [[ Hs | [Hsl Hs]] | [ Hsl1 [ Hsl2 Hs ]]].
  - destruct (fst (IHqr _ Hs _ (psize pi1 + psize pi2) (HR' _ Hs)) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
      as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
    exists (p, q, r), pi. lia.
  - apply Compare_dec.le_lt_eq_dec in Hsl as [ Hsl' | Hsl' ]; [ | subst qr ].
    + destruct (fst (IHqr _ Hsl' _ (psize pi1 + psize pi2) (HR' _ Hsl')) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
    + destruct (fst (IHb _ Hs _ (psize pi1 + psize pi2) eq_refl) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
  - apply Compare_dec.le_lt_eq_dec in Hsl1 as [ Hsl1' | Hsl1' ]; [ | subst qr ].
    + destruct (fst (IHqr _ Hsl1' _ (psize pi1 + psize pi2) (HR' _ Hsl1')) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
    + apply Compare_dec.le_lt_eq_dec in Hsl2 as [ Hsl2' | Hsl2' ].
      * destruct (fst (IHb _ Hsl2' _ (psize pi1 + psize pi2) eq_refl) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
          as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
        exists (p, q, r), pi. lia.
      * symmetry in Hsl2'.
        destruct (fst (IHs _ Hs _ Hsl2') _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p q] r] [pi Hqr]];
          [ reflexivity .. | ].
        exists (p, q, r), pi. lia. }

assert (forall a b' c l1 l2 p1 p2 q1 q2 r1 r2 (pi1 : fsub R p1 q1 r1 a l1 b') (pi2 : fsub R p2 q2 r2 b' l2 c),
         ((vsize pi1 + vsize pi2 < qr)%type
        + (vsize pi1 + vsize pi2 <= qr /\ fsize b' < fsize b)
        + (vsize pi1 + vsize pi2 <= qr /\ fsize b' <= fsize b /\ psize pi1 + psize pi2 < s)) ->
        {'(p, q, r) & { pi : fsub R p q r a (l1 ++ l2) c & vsize pi <= vsize pi1 + vsize pi2 }})
  as IH2.
{ intros a b' c l1 l2 p1 p2 q1 q2 r1 r2 pi1 pi2 [[ Hs | [Hsl Hs]] | [ Hsl1 [ Hsl2 Hs ]]].
  - destruct (snd (IHqr _ Hs _ (psize pi1 + psize pi2) (HR' _ Hs)) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
      as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
    exists (p, q, r), pi. lia.
  - apply Compare_dec.le_lt_eq_dec in Hsl as [ Hsl' | Hsl' ]; [ | subst qr ].
    + destruct (snd (IHqr _ Hsl' _ (psize pi1 + psize pi2) (HR' _ Hsl')) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
    + destruct (snd (IHb _ Hs _ (psize pi1 + psize pi2) eq_refl) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
  - apply Compare_dec.le_lt_eq_dec in Hsl1 as [ Hsl1' | Hsl1' ]; [ | subst qr ].
    + destruct (snd (IHqr _ Hsl1' _ (psize pi1 + psize pi2) (HR' _ Hsl1')) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
        as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
      exists (p, q, r), pi. lia.
    + apply Compare_dec.le_lt_eq_dec in Hsl2 as [ Hsl2' | Hsl2' ].
      * destruct (snd (IHb _ Hsl2' _ (psize pi1 + psize pi2) eq_refl) _ _ _ _ _ _ _ _ _ _ _ pi1 pi2)
          as [[[p q] r] [pi Hqr]]; [ reflexivity .. | ].
        exists (p, q, r), pi. lia.
      * symmetry in Hsl2'.
        destruct (snd (IHs _ Hs _ Hsl2') _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p q] r] [pi Hqr]];
          [ reflexivity .. | ].
        exists (p, q, r), pi. lia. }

assert (0 < qr -> forall x y, isub x y -> fsub_0 R δ( x) δ( y)) as Hvt.
{ intros Hqr x y ipi. inversion_clear ipi. clear X. induction X0.
  - rewrite (HR Hqr). apply var_var_unfold, r.
  - reflexivity.
  - assert (Hs1 := vsize_min_max IHX0_1).
    assert (Hs2 := vsize_min_max IHX0_2).
    destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ _ IHX0_1 IHX0_2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    assert (Hs := vsize_min_max pi).
    assert (p' = 0) as -> by lia.
    assert (q' = 0) as -> by lia.
    assert (r' = 0) as -> by lia.
    exact pi. }
```

```
clear IHqr IHb IHs HR'. split.

(* cut1 *)
- intros a c d l1 l2 p1 p2 q1 q2 r1 r2 pi1 pi2 Hqr Hs.
  remember (l1 ++ b :: l2) as l eqn:Heql. destruct pi2; try subst l; cbn in *.
  (* */omega_right *)
  + exists (0, 0, 0), omega_right. cbn. lia.
  (* */inter_left1 *)
  + destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (p', q', r'), (inter_left1 pi). cbn. lia.
  (* */inter_left2 *)
  + destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (p', q', r'), (inter_left2 pi). cbn. lia.
  (* */inter_right *)
  + destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_1) as [[[p1' q1'] r1'] [pil Hpil]]; [ intuition lia | ].
    destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_2) as [[[p2' q2'] r2'] [pir Hpir]]; [ intuition lia | ].
    exists (min p1' p2', max q1' q2', max r1' r2'), (inter_right pil pir). cbn. lia.
  (* */arrow_left *)
  + destruct l1 as [ | ? l1 ]; cbn in Heql; injection Heql as [= -> ->].
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_1) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
      exists (p' + p2, q' + q2, r' + r2), (arrow_left pi pi2_2). cbn in *. lia.
    * destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
      exists (p0 + p', q0 + q', r0 + r'), (arrow_left pi2_1 pi). cbn. lia.
  (* */arrow_right *)
  + revert pi2 Hqr Hs. rewrite <- app_assoc, <- app_comm_cons. intros pi2 Hqr Hs.
    destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    revert pi Hpi. rewrite app_comm_cons, app_assoc. intros pi Hpi.
    exists (p', q', r'), (arrow_right pi). cbn. lia.
  (* */var_rel *)
  + destruct l1; discriminate Heql.

  (* */var_right *)
  + destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (p', q', S r'), (var_right pi). cbn. lia.
  (* */var_left *)
  + destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (S p', S q', r'), (var_left pi). cbn. lia.

(* cut2 *)
- intros a c l1 l2 p1 p2 q1 q2 r1 r2 pi1 pi2 Hqr Hs.
  destruct pi2; cbn in *.
  (* */omega_right *)
  + exists (0, 0, 0), omega_right. cbn. lia.
  (* */inter_left1 *)
  + destruct (inv_inter_right pi1)
      as [[[[pl ql] rl] [pil Hpl Hvl]] [[[pr qr'] rr] [pir Hpr Hvr]]].
    destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pil pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (p', q', r'), pi. lia.
  (* */inter_left2 *)
  + destruct (inv_inter_right pi1)
      as [[[[pl ql] rl] [pil Hpl Hvl]] [[[pr qr'] rr] [pir Hpr Hvr]]].
    destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pir pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    exists (p', q', r'), pi. lia.
  (* */inter_right *)
  + destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_1) as [[[p1' q1'] r1'] [pi1' Hpi1]]; [ intuition lia | ].
    destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi2_2) as [[[p2' q2'] r2'] [pi2' Hpi2]]; [ intuition lia | ].
    exists (min p1' p2', max q1' q2', max r1' r2'), (inter_right pi1' pi2'). cbn. lia.
  (* */arrow_left *)
  + destruct (inv_arrow_right pi1) as [[[p' q'] r'] [pi' Hp Hvl]].
    destruct (IH2 _ _ _ _ _ _ _ _ _ _ pi' pi2_2) as [[[p'' q''] r''] [pi'' Hpi]]; [ intuition lia | ].
    revert pi'' Hpi. rewrite <- app_assoc. cbn. intros pi'' Hpi.
    destruct (IH1 _ _ _ _ _ _ _ _ _ _ _ pi2_1 pi'') as [[[p''' q'''] r'''] [pi''' Hpi']]; [ intuition lia | ].
    exists (p''', q''', r'''), pi'''. lia.
  (* */arrow_right *)
  + destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi Hpi]]; [ intuition lia | ].
    revert pi Hpi. rewrite app_assoc. intros pi Hpi.
    exists (p', q', r'), (arrow_right pi). cbn. lia.
  (* */var_rel *)
  + remember (var x) as b eqn:Hx.
    destruct pi1; destr_eq Hx; cbn in *; try match type of Hx with ?z = _ => subst z end.
    (* inter_left1/var_rel *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_rel r)) as [[[p' q'] r'] [pi Hpi]]; [ cbn; intuition lia | ].
      exists (p', q', r'), (inter_left1 pi). cbn in *. lia.
    (* inter_left2/var_rel *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_rel r)) as [[[p' q'] r'] [pi Hpi]]; [ cbn; intuition lia | ].
      exists (p', q', r'), (inter_left2 pi). cbn in *. lia.
    (* arrow_left/var_rel *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1_2 (var_rel r)) as [[[p' q'] r'] [pi Hpi]];
        [ cbn; intuition lia | ].
      exists (p1 + p', q1 + q', r1 + r'), (arrow_left pi1_1 pi). cbn in *. lia.
    (* var_rel/var_rel *)
    * exists (0, 0, 0), (var_rel (HRt.(PreOrder_Transitive) _ _ _ r0 r)). cbn. lia.
    (* var_right/var_rel *)
    * assert (0 < qr) as Hqr' by lia.
      revert pi1 Hs Hqr. rewrite (HR Hqr') in *. intros pi1 Hs Hqr.
      assert (pi := Hvt ltac:(lia) _ _ r).
      assert (Hpi := vsize_min_max pi).
      destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi) as [[[p' q'] r'] [pi' Hpi']]; [ intuition lia | ].
      exists (p', q', S r'), (var_right pi'). cbn. lia.
    (* var_left/var_rel *)
```

```
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_rel r)) as [[[p' q'] r'] [pi Hpi]]; [ cbn; intuition lia | ].
      exists (S p', S q', r'), (var_left pi). cbn in *. lia.
  (* */var_right *)
  + destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi' Hpi]]; [ intuition lia | ].
    exists (p', q', S r'), (var_right pi'). cbn. lia.
  (* */var_left *)
  + remember (var x) as b eqn:Hx.
    destruct pi1; destr_eq Hx; cbn in *; match type of Hx with ?z = _ => subst z end.
    (* inter_left1/var_left *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_left pi2)) as [[[p' q'] r'] [pi Hpi]];
        [ cbn; intuition lia | ].
      exists (p', q', r'), (inter_left1 pi). cbn in *. lia.
    (* inter_left2/var_left *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_left pi2)) as [[[p' q'] r'] [pi Hpi]];
        [ cbn; intuition lia | ].
      exists (p', q', r'), (inter_left2 pi). cbn in *. lia.
    (* arrow_left/var_left *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1_2 (var_left pi2)) as [[[p' q'] r'] [pi Hpi]];
        [ cbn; intuition lia | ].
      exists (p1 + p', q1 + q', r1 + r'), (arrow_left pi1_1 pi). cbn in *. lia.
    (* var_rel/var_left *)
    * assert (0 < qr) as Hqr' by lia.
      revert pi2 Hs Hqr. rewrite (HR Hqr') in *. intros pi2 Hs Hqr.
      assert (pi := Hvt ltac:(lia) _ _ r0).
      assert (Hpi := vsize_min_max pi).
      destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi pi2) as [[[p' q'] r'] [pi' Hpi']]; [ intuition lia | ].
      exists (S p', S q', r'), (var_left pi'). cbn in *. lia.
    (* var_right/var_left *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 pi2) as [[[p' q'] r'] [pi' Hpi']]; [ intuition lia | ].
      exists (p', q', r'), pi'. lia.
    (* var_left/var_left *)
    * destruct (IH2 _ _ _ _ _ _ _ _ _ _ _ pi1 (var_left pi2)) as [[[p' q'] r'] [pi Hpi]];
        [ cbn; intuition lia | ].
      exists (S p', S q', r'), (var_left pi). cbn in *. lia.
Qed.
```